

TrashUQ: Federated Learning for Intelligent Waste Monitoring on Arduino UNO Q

Aleix BERTRAN ANDREU^a Pol LLINÀS VAQUER^a Bru PALLÀS VARGUÉS^a
Aleix ROSINACH OLIVART^a

^a *Universitat de Lleida, Lleida, Catalonia*

Abstract. This paper presents TrashUQ, a waste-monitoring platform that integrates Arduino UNO Q edge nodes, MQTT telemetry, and a gRPC Federated Learning (FL) coordinator. We report two validations. First, a real two-device deployment confirms end-to-end telemetry and FL round-trip behavior (282 persisted MQTT records, model version 0→26, 25/30 successful aggregations). Second, a controlled non-IID FL simulation (Dirichlet $\alpha = 0.3$) with 2, 5, 10, and 20 clients shows stable FedAvg convergence over 25 rounds, with final accuracies around 93–94%. We further demonstrate that stochastic rounding gradient quantization achieves 4–8× model compression with negligible accuracy degradation (0–1.33%). Results indicate that FL orchestration is feasible on this embedded stack, and that gradient quantization effectively mitigates the communication bottleneck.

Keywords. Federated Learning, Edge AI, Waste Classification, Arduino UNO Q, MQTT, TrashNet

1. Introduction

Waste mismanagement poses critical environmental and public health challenges. Intelligent waste sorting systems powered by deep learning can significantly improve recycling rates, but conventional approaches rely on centralized cloud infrastructure that raises privacy, latency, and bandwidth concerns [1]. Federated Learning (FL) addresses these limitations by training models collaboratively across edge devices while keeping data local [2,3].

Recent works have shown mature FL methods and system-level considerations for heterogeneous environments [3]. However, most existing solutions focus on simulation-based evaluation and lack validation on real heterogeneous hardware. The availability of low-cost edge AI platforms such as the Arduino UNO Q—featuring a Qualcomm QRB2210 processor and an STM32U585 co-processor—enables practical FL deployment on physical devices.

This paper presents TrashUQ, a complete FL-based waste monitoring system deployed on two Arduino UNO Q boards with heterogeneous roles. Our contributions are:

1. A real FL deployment on two Arduino UNO Q boards, including a camera-equipped capture-and-classify node and a compute-only FL participant, with multi-node telemetry validation and FL round-trip timing.

2. FL simulation results on a controlled synthetic TrashNet-style dataset with FedAvg across 2, 5, 10, and 20 non-IID clients, demonstrating convergence behavior and communication efficiency.
3. An open-source implementation of the complete FL pipeline, from on-device fine-tuning to MQTT telemetry, publicly available at <https://github.com/TrashUQ/TrashUQ> [4].

2. Related Work

Communication efficiency is a central challenge in federated learning, where the cost of transmitting model updates between clients and the server can dominate training time [3]. Several compression strategies have been proposed to mitigate this bottleneck.

Alistarh et al. [5] introduced QSGD, which combines stochastic gradient quantization with unbiased encoding to achieve convergence guarantees under bounded variance. Bernstein et al. [6] proposed SignSGD, where only the sign of each gradient is communicated, enabling 1-bit compression with majority-vote aggregation. Haddadpour et al. [7] developed FedCOM, providing a unified analysis of compressed FL with periodic communication and local gradient tracking. Top-k sparsification [8] further reduces communication by transmitting only the largest gradient components. Hierarchical FL approaches [9,10] introduce multi-level aggregation to reduce coordinator load in large-scale deployments.

However, most prior work evaluates these techniques exclusively in simulation environments with high-end hardware, leaving open questions about their applicability to resource-constrained edge deployments. In this work, we implement stochastic rounding quantization within a simulated FedAvg pipeline and validate it on synthetic non-IID partitions, bridging the gap between theoretical compression and practical edge deployment.

3. System Architecture

3.1. Hardware Platform

Each Arduino UNO Q has a Qualcomm QRB2210 application processor (Debian Linux) and an STM32U585 microcontroller. Boards connect to the backend over Wi-Fi.

3.2. Edge Software Pipeline

The edge daemon follows capture/classify/act logic. Classification uses a MobileNetV2 TFLite model (224×224 , classes: cardboard, glass, paper, plastic). User corrections are stored locally and used for on-device fine-tuning.

3.3. Federated Learning Pipeline

The trainable model is a frozen MobileNetV2 backbone with a small learnable head. After accumulating new labels, a node starts an FL round: it fetches global weights, trains locally, and submits an update via gRPC. The coordinator applies FedAvg [1].

3.4. MQTT Telemetry and Backend

Boards publish status/events/FL metrics to `arduino/<device-id>/#`. A FastAPI backend persists telemetry in PostgreSQL and exposes the FL coordinator. A Next.js dashboard provides real-time monitoring via MQTT over WebSocket. Code and deployment artifacts are available at <https://github.com/TrashUQ/TrashUQ>.

4. Hardware Validation

The hardware validation was executed on May 16, 2026, using two Arduino UNO Q nodes connected to a shared backend stack (MQTT broker, PostgreSQL, REST API, dashboard, and gRPC FL coordinator). Both nodes ran on Debian 13 and used the same edge daemon, topic contract, and FL client API.

4.1. Protocol Validation and Persistence

We validated end-to-end ingestion for both devices across the path Edge $\rightarrow$ MQTT $\rightarrow$ Backend $\rightarrow$ PostgreSQL $\rightarrow$ Frontend. During a $\sim$ 12-minute run, the system persisted 282 MQTT rows in total: 147 from `unoq-01` and 135 from `unoq-02`. Per-device topic counts were 27/21 status messages, 16/14 metric messages, and 104/100 event messages, respectively.

4.2. FL Round-Trip Behavior on Real Nodes

Both nodes successfully completed FL handshake and training-update cycles with the coordinator (`Join`, `GetGlobalModel`, `SubmitUpdate`). The experiment reached model version 26, with 25 successful aggregations out of 30 submissions (83.3% aggregation success). Five submissions were rejected as stale during concurrent updates, confirming that coordinator-side monotonic round checks were enforced correctly.

Round submission latency (from local training completion to update submission) was low (median $\sim$ 25 ms, max 105 ms). Mean local metrics were 0.921 accuracy / 0.236 loss for `unoq-01`, and 0.871 accuracy / 0.311 loss for `unoq-02`.

4.3. Operational Limitations

Both devices used `-fake-camera` and HTTP label injection to exercise FL deterministically. Therefore, camera-in-the-loop inference latency and classification-topic throughput were not measured in this run.

5. FL Simulation and Communication Efficiency

Part B provides scale-oriented FL evidence under controlled simulation. We ran FedAvg for 25 global rounds with non-IID client partitions generated by Dirichlet sampling ($\alpha = 0.3$), using 3 seeds per setting and client counts $\{2, 5, 10, 20\}$.

Table 1. FL simulation summary (mean values across 3 seeds).

Clients	Final Acc. (%)	Final Loss	Round Time (s)	Total Comm. (MB)
2	93.75	0.2192	0.005	0.394
5	93.56	0.2004	0.005	0.985
10	93.37	0.2018	0.006	1.970
20	93.75	0.1933	0.008	3.941

5.1. Synthetic Dataset

To enable reproducible experiments without external dataset dependencies, we generated a synthetic dataset of 16×16 grayscale images spanning four waste categories (cardboard, glass, paper, plastic). Each class is defined by a distinct geometric prototype: horizontal stripes for cardboard, diagonal center-line patterns for glass, nested rectangular regions for paper, and vertical stripes for plastic. Individual samples are produced by a convex combination of the primary prototype (52%), the next-class prototype (23%), an alternate-class prototype (10%), and the mean shared pattern (15%), followed by additive Gaussian noise ($\sigma = 0.30$), brightness jitter (8%), stochastic pixel roll (35% probability), and occasional class blending (12–17%). This process yields 220 samples per class while maintaining visual separability, enabling reproducible non-IID partitions via Dirichlet sampling. The simulator can optionally load real TrashNet images if the corresponding directory structure is provided.

5.2. Configuration

Each run used local epochs = 2, batch size = 16, SGD with learning rate 0.18, and full client participation per round.

5.3. Convergence Results

Across all settings, FL converged stably with final global accuracy near 93–94% and no failed rounds. Mean final accuracy was 93.75% (2 clients), 93.56% (5), 93.37% (10), and 93.75% (20). Mean final loss ranged from 0.2192 (2 clients) down to 0.1933 (20 clients). Exact-permutation tests on final accuracy differences (2 vs 5, 5 vs 10, 10 vs 20) were not significant at $p < 0.05$.

5.4. Efficiency and Communication Trade-Off

Mean round duration increased with scale from 0.005 s (2 clients) to 0.008 s (20 clients), while communication grew approximately linearly with client count. Total communication (sent+received) increased from 0.394 MB (2 clients) to 3.941 MB (20 clients). Consequently, accuracy-per-MB decreased from 2.38 to 0.24, illustrating the expected cost-accuracy trade-off as participation expands.

These results support the main claim of the paper: the FL workflow remains robust under increasing non-IID client heterogeneity, but communication efficiency becomes the limiting factor at larger scales.

Table 2. Quantization results summary (20 clients, mean across 3 seeds).

Config	Final Acc. (%)	Final Loss	Total Comm. (MB)	Compression
float32 (baseline)	93.75	0.1933	3.941	1.0×
8-bit	93.75	0.1935	2.474	~4× weights
4-bit packed	93.75	0.1976	2.229	~8× weights

5.5. Gradient Quantization

To address the communication bottleneck, we integrate stochastic rounding quantization into the FedAvg pipeline. After local training, each client quantizes its weight updates to a target bit width before transmission; the server dequantizes the received updates before weighted averaging. We evaluate two configurations: 8-bit (4× weight compression) and 4-bit with uint8 packing (2 values per byte, 8× weight compression). The total communication volume includes protocol overhead, so the observed reduction is approximately 1.59× for 8-bit and 1.77× for 4-bit packed at 20 clients.

The 8-bit configuration preserves accuracy identically (93.75%) while reducing communication by approximately 1.59× total (4× model compression). The 4-bit packed variant achieves approximately 1.77× total communication reduction (8× model compression) with a maximum accuracy degradation of 1.33% observed at 2 clients, which diminishes to zero as the client population grows. These results demonstrate that lightweight gradient quantization is effective on resource-constrained edge hardware without compromising model quality.

6. Conclusion and Future Work

TrashUQ combines real two-node Arduino UNO Q validation with controlled FL simulation. Real deployment confirmed persistent multi-node telemetry and FL coordination up to model version 26; simulation confirmed stable non-IID FedAvg convergence across 2–20 clients with final accuracy around 93–94%. We further demonstrate that stochastic rounding gradient quantization provides 4–8× model compression with negligible accuracy loss (0–1.33%), effectively mitigating the communication bottleneck identified in prior sections. Code is available at <https://github.com/TrashUQ/TrashUQ>; future work will focus on camera-in-the-loop latency validation and deployment of quantized models on Arduino UNO Q hardware.

Acknowledgements

The authors thank Jordi Planes Cid from the Universitat de Lleida for guidance and support during this work.

References

- [1] McMahan HB, Moore E, Ramage D, Hampson S, y Arcas BA. Communication-Efficient Learning of Deep Networks from Decentralized Data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS); 2017. p. 1273-82.

- [2] Li T, Sahu AK, Talwalkar A, Smith V. Federated Optimization in Heterogeneous Networks. In: Proceedings of Machine Learning and Systems (MLSys); 2020. .
- [3] Kairouz P, McMahan HB, Avent B, et al. Advances and Open Problems in Federated Learning. Foundations and Trends in Machine Learning. 2021;14(1–2):1-210.
- [4] Team T. TrashUQ Project Repository; 2026. <https://github.com/TrashUQ/TrashUQ>.
- [5] Alistarh D, Grubic D, Li J, Tomioka R, Vojnovic M. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. In: Advances in Neural Information Processing Systems (NeurIPS); 2017. p. 1709-20.
- [6] Bernstein J, Wang YX, Azizzadenesheli K, Anandkumar A. signSGD: Compressed Optimisation for Non-Convex Problems. In: Proceedings of the 35th International Conference on Machine Learning (ICML). vol. 80. PMLR; 2018. p. 560-9.
- [7] Haddadpour F, Kamani MM, Mokhtari A, Mahdavi M. Federated Learning with Compression: Unified Analysis and Sharp Guarantees. IEEE Transactions on Information Theory. 2021;67(7):4750-76.
- [8] Aji AF, Heafield K. Sparse Communication for Distributed Gradient Descent. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP); 2017. p. 440-50.
- [9] Liu L, Zhang J, Song SH, Letaief KB. Client-Edge-Cloud Hierarchical Federated Learning. In: IEEE International Conference on Communications (ICC); 2020. p. 1-6.
- [10] Abad MSH, Ozfatura E, Gunduz D, Ercetin O. Hierarchical Federated Learning Across Heterogeneous Cellular Networks. In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP); 2020. p. 8866-70.